

Resenha – The Hexagonal (Ports & Adapters) Architecture

"The Hexagonal (Ports & Adapters) Architecture" é um relatório técnico escrito por Alistair Cockburn e publicado em 2005. O texto apresenta um padrão arquitetural chamado Ports and Adapters, também conhecido como Arquitetura Hexagonal, cujo objetivo principal é isolar a lógica de negócio de uma aplicação de tudo que está ao redor dela, seja uma interface de usuário, um banco de dados ou qualquer outro sistema externo. É um texto curto e direto, escrito num tom de artigo técnico-prático, com exemplos de código e diagramas para ilustrar os conceitos.

O problema que motiva o padrão é um dos mais clássicos do desenvolvimento de software: a lógica de negócio que vaza para a camada de interface do usuário. Cockburn descreve que, ao longo dos anos, a solução recorrente era criar uma nova camada na arquitetura com a promessa de que nenhuma lógica de negócio seria colocada ali. O problema é que essa promessa nunca tinha como ser verificada automaticamente, e com o tempo a nova camada acabava tão poluída quanto a anterior. A consequência prática era que o sistema ficava impossível de testar de forma automatizada, não podia ser executado sem uma interface visual e não conseguia ser integrado com outros sistemas de forma programática.

A solução proposta é pensar a aplicação como um núcleo interno cercado de portas. Cada porta representa uma conversa com propósito definido, seja receber dados de um usuário, acessar um banco de dados ou se comunicar com outro sistema. Para cada porta, existem adaptadores, que são as implementações específicas de cada tecnologia que pode ser conectada ali. O ponto central é que a aplicação não sabe nada sobre quem está do outro lado da porta. Ela só conhece a interface da própria porta, o que significa que pode ser testada com adaptadores falsos, executada sem interface gráfica e integrada com qualquer tecnologia sem precisar mudar seu núcleo.

O hexágono do nome não tem nada de especial com o número seis. Cockburn explica que a forma foi escolhida por razões visuais e práticas: ela rompe com o desenho tradicional em camadas empilhadas, que induz as pessoas a pensar em cima e embaixo, dentro e fora, e ajuda a representar que pode haver múltiplas portas em lados diferentes da aplicação. O que importa de fato é a distinção entre dentro e fora, não entre esquerda e direita.

O autor também faz uma distinção importante entre portas e adaptadores primários e secundários. Os primários são os que dirigem a aplicação, como um usuário humano ou um script de teste. Os secundários são os que a aplicação aciona, como um banco de dados ou um serviço externo. Essa distinção tem impacto direto em como os testes são escritos: adaptadores de teste para o lado primário costumam ser frameworks como o FIT, que simulam uma entidade que dá comandos para o sistema. Já os adaptadores de teste para o lado secundário costumam ser mocks em memória, que fingem ser o banco de dados sem precisar que ele esteja disponível.

O exemplo de código apresentado no artigo é simples e funcional: um sistema de cálculo de desconto que recebe um valor do usuário e busca uma taxa num repositório. O Cockburn mostra o passo a passo de como construir isso em três estágios, começando com um mock constante no lugar do banco, depois adicionando uma interface gráfica e por fim substituindo o mock por uma interface real de repositório que pode ser trocada por qualquer implementação sem alterar a aplicação.

O ponto forte do artigo é a clareza conceitual. A ideia de tratar todos os lados da aplicação de forma simétrica, chamando tudo de porta, é elegante e resolve um problema real que a arquitetura em camadas não resolve bem. A limitação mais visível é que o texto não aprofunda cenários mais complexos, com muitas portas ou com questões de performance e concorrência. No geral, é uma leitura rápida e muito útil para quem quer entender por que separar lógica de negócio de infraestrutura não é só uma boa prática, mas uma decisão arquitetural com consequências concretas em testabilidade, manutenção e flexibilidade do sistema.